

# AWS Cloud Infrastructure Cost Optimization Report

Comparative Analysis of Hosting Scenarios & Workflow Architectures for SnapURL

|                                      |                                          |
|--------------------------------------|------------------------------------------|
| Prepared For: Prashant Bhushan       | Date: August 7, 2026                     |
| Active Credit Balance: \$88.12 USD   | Credit Days Remaining: 172 Days          |
| Current Burning Rate: ~\$35.37/month | Target Optimization Rate: ~\$21.87/month |

## Executive Summary

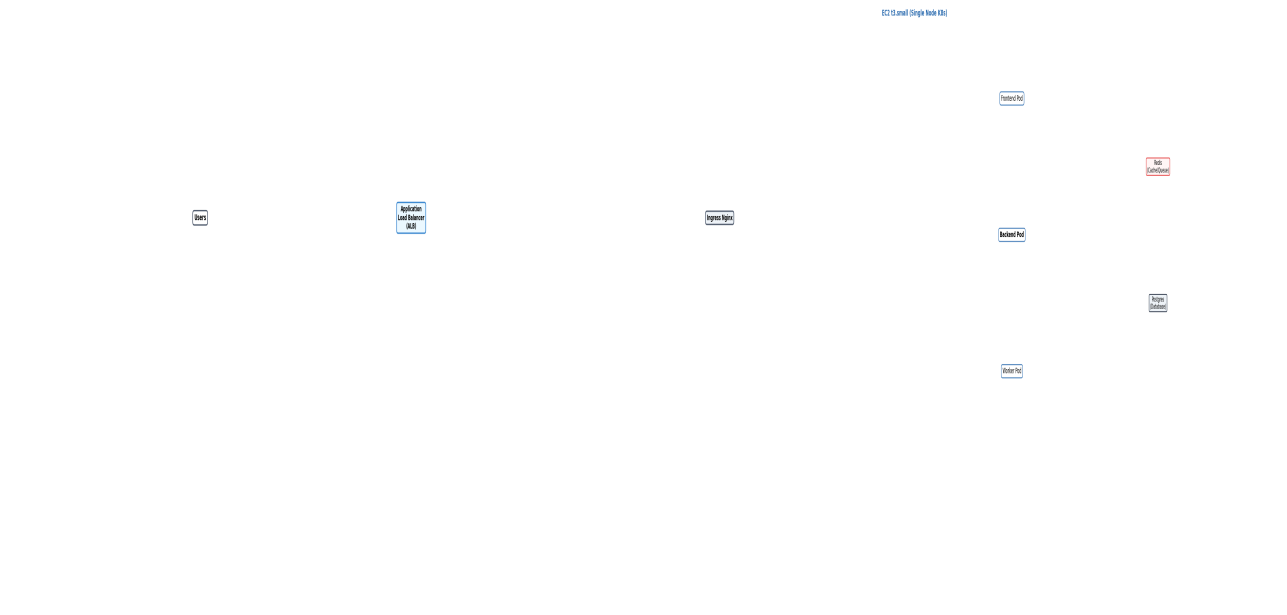
This report evaluates various AWS infrastructure hosting configurations for the SnapURL application. The goal is to maximize the utility of your remaining **\$88.12 AWS credits** (valid for 172 days). Currently, your single-node custom Kubernetes setup on an EC2 instance consumes **~\$35.37/month**, meaning credits will run dry in **~2.5 months**. By transitioning to an **Optimized Hybrid architecture**, we can reduce monthly charges to **~\$21.87/month**, successfully extending your credit lifespan to **4.0 months (120 days)**.

## AWS Infrastructure Scenarios & Workflow Architectures

### Scenario 1: EC2 + Custom Kubernetes (Current Setup)

Your current setup hosts all workloads (Frontend, Backend, Worker, Postgres, Redis) in Kubernetes on a single EC2 t3.small instance. External traffic is routed through an Application Load Balancer (ALB).

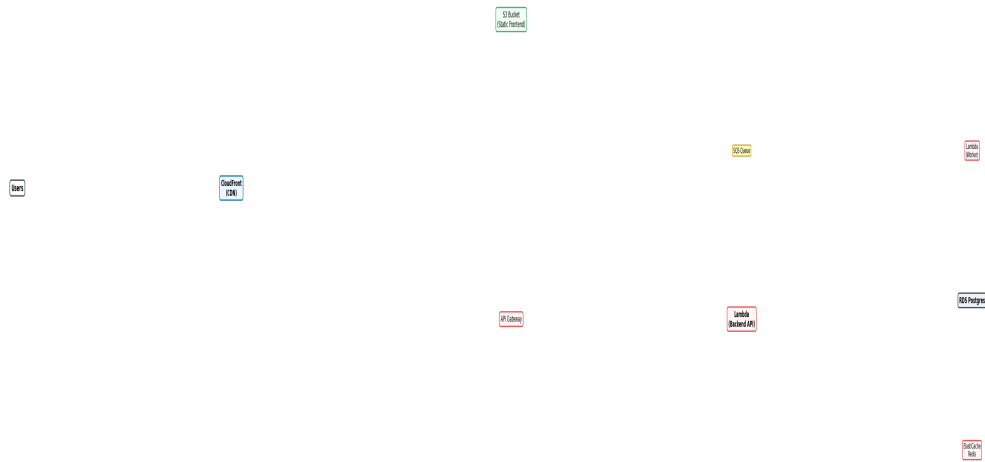
- **Cost:** ~\$35.37/month (ALB and Public IP represent 50% of the cost).
- **Pros:** Fully operational right now, zero code refactoring.
- **Cons:** Redundant ALB expense, tight RAM limit on node (~1.6GB of 1.9GB used).



## Scenario 2: Fully Serverless (Lambda + S3 + RDS + ElastiCache)

Transitioning the application to an event-driven serverless system. The static frontend resides on S3/CloudFront, APIs run on AWS Lambda behind an API Gateway, and database/cache are hosted on managed RDS and ElastiCache.

- **Cost:** ~\$30.00 - \$33.00/month (RDS and ElastiCache are always-on and have high baseline costs).
- **Pros:** Seamless auto-scaling, high availability, zero server maintenance.
- **Cons:** Requires heavy refactoring of backend/worker; cold starts cause 1-3 sec API delays; RDS Postgres cost is similar to the EC2 host.



### Scenario 3: Managed EKS + RDS + ElastiCache

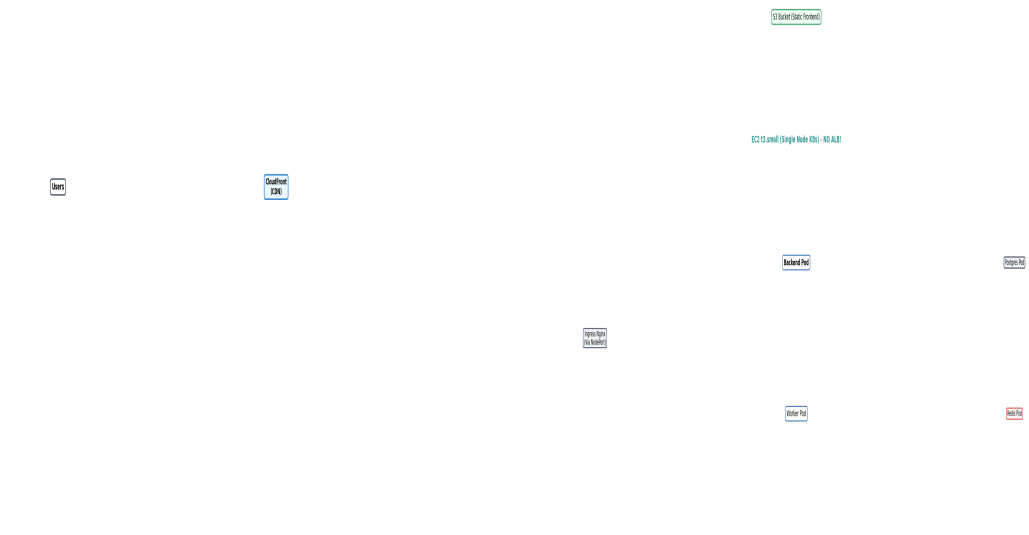
A standard enterprise Kubernetes deployment using AWS Elastic Kubernetes Service (EKS) for orchestration, with Postgres and Redis running on managed RDS and ElastiCache.

- **Cost:** ~\$131.37/month (EKS Control Plane fee is \$73/mo).
- **Pros:** Production-grade reliability, offloads control-plane RAM from worker nodes.
- **Cons:** Prohibitively expensive. Your credits will expire in less than 20 days.

### Scenario 4: Optimized Hybrid (Recommended)

An optimized setup tailored specifically for cost efficiency. The static frontend is deployed to S3 + CloudFront CDN. The backend APIs, worker, Postgres, and Redis remain on the EC2 t3.small instance. Crucially, the **Application Load Balancer (\$14/mo) is removed**, and CloudFront routes API requests directly to a NodePort or HostPort Ingress on the EC2 instance.

- **Cost:** ~\$21.87/month (Saves ~38% compared to Scenario 1).
- **Pros:** Slashes costs, removes ALB overhead, extends credit lifespan, keeps deployment simple.
- **Cons:** Minor configuration changes to point CloudFront to the EC2 backend API.



### Side-by-Side Comparison Matrix

| Comparison Factor | Scenario 1<br>EC2 + K8s | Scenario 2<br>Serverless | Scenario 3<br>EKS Managed | Scenario 4<br>Optimized Hybrid |
|-------------------|-------------------------|--------------------------|---------------------------|--------------------------------|
| Monthly Cost      | ~\$35.37                | ~\$31.00                 | ~\$131.37                 | ~\$21.87 (WINNER)              |
| Credit Lifespan   | 2.5 Months              | 2.8 Months               | < 1 Month ■               | 4.0 Months (120 days)          |
| Auto-Scaling      | Manual / HPA limits     | Instant Serverless       | Excellent Elasticity      | Manual / HPA limits            |
| Cold Starts       | None (Always On)        | 1-3 Seconds              | None (Always On)          | None (Always On)               |
| Code Changes      | None (Already setup)    | Heavy Refactoring        | Minor Adjustments         | None (Infra only)              |
| Complexity        | Medium                  | High                     | High                      | Low-Medium                     |

## Deep Dive: Recommended Hybrid Scenario (Scenario 4)

By using S3 and CloudFront for the frontend assets, we take the heavy lifting of serving static files (HTML, CSS, JS, Images) off the EC2 instance. More importantly, we eliminate the costly Application Load Balancer (ALB) and route user requests directly to your EC2 instance via CloudFront. Here is the exact cost listing for Scenario 4:

| AWS Component                          | Used For                                               | Monthly Cost                     |
|----------------------------------------|--------------------------------------------------------|----------------------------------|
| <b>S3 Bucket</b>                       | Hosting Frontend Static Files (HTML/JS/CSS)            | \$0.02 (Est.)                    |
| <b>CloudFront CDN</b>                  | Global distribution & Routing HTTPS to EC2             | \$0.50 (Est. based on bandwidth) |
| <b>EC2 t3.small</b>                    | Hosting K8s pods (Backend, Worker, Postgres, Redis)    | \$15.77                          |
| <b>EBS Volume (22GB GP3)</b>           | Persistent database disk space (Postgres/Redis data)   | \$2.00                           |
| <b>AWS Public IP</b>                   | Required for EC2 to be reachable by CloudFront         | \$3.60                           |
| <b>Application Load Balancer (ALB)</b> | <b>REMOVED (Replaced by direct Nginx/Node ingress)</b> | <b>\$0.00 (Saved \$14.00!)</b>   |
| <b>Total Monthly Cost</b>              | <b>Optimized Hybrid Infrastructure</b>                 | <b>~\$21.89 / Month</b>          |

## Implementation Plan (How to Migrate)

To safely transition to the Optimized Hybrid scenario, follow these three stages:

### Stage 1: Build & Deploy Frontend to S3 + CloudFront

1. Build your frontend code locally or in CI (e.g. `npm run build`) to generate static files.
2. Create an AWS S3 bucket, enable static website hosting, and upload the build directory files.
3. Set up a CloudFront distribution pointing to the S3 bucket as the default origin.

### Stage 2: Configure Routing for Backend APIs

1. Configure CloudFront with an additional Origin pointing directly to your EC2 instance's public IP address.
2. Create a Cache Behavior in CloudFront: Send any traffic matching `/api/*` to the EC2 Backend origin (bypassing S3 caching).
3. Configure NodePort or a HostPort ingress controller on your EC2 single-node Kubernetes cluster to accept incoming traffic on port 80/443 without an ALB.

### Stage 3: Remove ALB and Clean Up

1. Point your domain name (e.g. via Route53) to the CloudFront distribution instead of the ALB.
2. Test both frontend and backend functionality thoroughly to verify API communication works via CloudFront.
3. Delete the Application Load Balancer (ALB) from your AWS EC2 Console to stop the \$14.00/month recurring charge.